University of Padua

Bachelor's Degree in Computer Engineering

Lecture notes and laboratory handbook

Concurrent Systems Engineering

Professor

Prof. Ada Lovelace

Academic year

2026–2027

Degree year

1

Semester

1

Author

Simone Siega

Date

3 August 2026

Version

1.0.0

Table of contents

1 A bounded-buffer model

A bounded buffer decouples producers from consumers without allowing memory usage to grow indefinitely. The implementation considered here stores at most N elements and protects every state transition with one mutex. Two counting semaphores record the number of occupied and free positions. This small model is representative of network queues, device drivers, and streaming applications.

Definition 1.1. A buffer state is a triple (h, t, n), where h is the next position to read, t is the next position to write, and n is the number of stored elements. Indices are interpreted modulo the capacity N.

Theorem 1.1. *Every execution that initializes $n = 0$ and updates the counter while holding the mutex preserves*

$$0 \le n \le N.$$

Proof. An insertion is enabled only when a free-position permit has been acquired, so it starts with $n < N$ and changes the counter to $n + 1 \le N$. A removal similarly requires an occupied-position permit, starts with $n > 0$, and changes the counter to $n - 1 \ge 0$. Mutual exclusion serializes these updates; induction over the execution therefore proves the invariant. □

On the laboratory machine, an uncontended critical section takes 1.5 µs, while the measured wake-up samples are 8.2, 8.7 and 9.1µs. These values suggest keeping data preparation outside the mutex and limiting the protected region to index and counter updates.

Theorem 1.2. *If insertion writes at t before advancing t, and removal reads at h before advancing h, the buffer returns elements in first-in, first-out order.*

Proof. Both indices advance by one modulo N. A position cannot be reused until its element has been removed because theorem 1.1 prevents the producer from overtaking the consumer. Consequently, removal visits occupied positions in the same order in which insertion filled them. □

The safety properties do not by themselves guarantee progress: an unfair scheduler may postpone a ready thread indefinitely. In practice, the semaphore implementation and the operating-system scheduler determine whether waiting threads receive service with acceptable latency.

2 Architecture and measurements

Figure 2.1 shows the data path. Producers perform expensive parsing before entering the critical section; consumers release the occupied slot before processing the item. The ownership rule is simple: the queue owns an item from the successful insertion until the corresponding removal returns it.

Figure 2.1. Ownership transfer through the bounded buffer

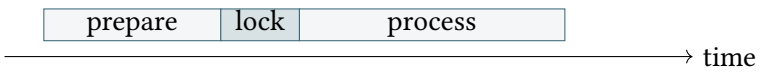

Figure 2.2. Critical-section placement in one transaction

The experiment uses a fixed workload of one million messages after a warm-up interval. Each reported value is the median of seven runs, and throughput is computed from completed removals rather than attempted insertions. Table 2.1 separates useful capacity from synchronization overhead.

Table 2.1. Throughput as the buffer capacity changes

Capacity	Throughput (messages/s)	99th-percentile latency
1	82000	41 µs
16	241000	18 µs
64	255000	19 µs
256	254000	27 µs

Table 2.2. State transitions of the queue operations

Operation	Precondition	Atomic state change
Insert	$n < N$	$a[t] \leftarrow x, t \leftarrow (t+1) \bmod N, n \leftarrow n+1$
Remove	$n > 0$	$x \leftarrow a[h], h \leftarrow (h+1) \bmod N, n \leftarrow n-1$
Inspect	Mutex held	Read (h, t, n) without modification

```
def insert(queue, item):
    with queue.mutex:
        if queue.count == queue.capacity:
            raise BufferError("buffer is full")
        queue.data[queue.tail] = item
        queue.tail = (queue.tail + 1) % queue.capacity
        queue.count += 1
```

Listing 2.1. A checked insertion operation

3 Scheduling and verification

A blocking insertion first reserves one free position and then acquires the mutex. Reversing these operations can deadlock: a producer may hold the mutex while waiting for a consumer that needs the same mutex to remove an item. The removal operation used by the test harness is shown in listing 3.1.

```
def remove(queue):
    with queue.mutex:
        if queue.count == 0:
            raise BufferError("buffer is empty")
        item = queue.data[queue.head]
        queue.head = (queue.head + 1) % queue.capacity
        queue.count -= 1
        return item
```

Listing 3.1. A checked removal operation

The checks in listings 2.1 and 3.1 turn invariant violations into immediate failures during testing. Production code normally waits on a semaphore, but assertions around the internal counter still detect incorrect signaling and accidental unlocked access. Algorithm 3.1 uses the safe acquisition order.

Input: A queue q and an item x
Output: The updated queue

```
wait(q.free)
lock(q.mutex)
q.data[q.tail] ← x
q.tail ← (q.tail + 1) mod q.capacity
unlock(q.mutex)
signal(q.used)
return q
```

Algorithm 3.1. Blocking insertion

The validation pass in algorithm 3.2 is intentionally read-only. It can run after each randomized operation in a model-based test, comparing the implementation state with a simple reference sequence.

Input: A queue state q
Output: True exactly when the state is valid

```
if q.count < 0 or q.count > q.capacity then
    return false
end
for i ← 1 to q.count do
    check the element at (q.head + i − 1) mod q.capacity
end
return true
```

Algorithm 3.2. Queue-state validation

Lowercase singular references are demonstrated by theorem 1.1, figure 2.1, table 2.1, listing 2.1, and algorithm 3.1. Capitalized singular forms are Theorem 1.1, Figure 2.1, Table 2.1, Listing 2.1, and Algorithm 3.1.

Lowercase plural forms are theorems 1.1 and 1.2, figures 2.1 and 2.2, tables 2.1 and 2.2, listings 2.1 and 3.1, and algorithms 3.1 and 3.2. Capitalized plural forms are Theorems 1.1 and 1.2, Figures 2.1 and 2.2, Tables 2.1 and 2.2, Listings 2.1 and 3.1, and Algorithms 3.1 and 3.2.

Document lists

List of Figures

List of Tables

List of algorithms

List of listings

A Laboratory checklist

Before collecting measurements, build the program with optimization enabled but assertions retained. Pin worker threads when comparing synchronization strategies, record the processor model and operating-system version, and keep the input workload identical. Run a warm-up phase to populate caches and stabilize dynamic frequency scaling.

For each operation, verify the invariant $0 \le n \le N$, compare the removed sequence with the reference model, and execute tests with capacities 1, 2, and a non-power-of-two value. Capacity one exposes ordering mistakes; wraparound-heavy tests expose incorrect index arithmetic.

A complete report includes the source revision, compiler flags, number of repetitions, aggregation method, and raw observations. It distinguishes throughput from latency and explains outliers rather than deleting them silently. These practices make the experiment reproducible and keep performance conclusions tied to the implementation that produced them.